

LAPORAN MODUL 8 PRAKTIKUM ALGORITMA DAN PEMROGRAMAN LANJUTAN



Disusun oleh :

Nama : Irvan Cahya Nugraha
NIM : 245410034
Kelas : Informatika 1

**PROGRAM STUDI INFORMATIKA
PROGRAM SARJANA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS TEKNOLOGI DIGITAL INDONESIA
YOGYAKARTA
2024 / 2025**

MODUL 8

METHOD

A. TUJUAN PRAKTIKUM

1. Mahasiswa dapat memahami, membuat dan menyelesaikan kasus dengan menggunakan method tanpa parameter

B. DASAR TEORI

1. Definisi Method / Function

Fungsi digunakan untuk mempermudah didalam membuat sebuah program, terutama program yang besar dan banyak melakukan beberapa hal yang sama. Fungsi memiliki ciri-ciri sebagai berikut:

1. Memiliki nama dari fungsi tersebut.
2. Memiliki tugas spesifik tertentu.
3. Memiliki sekumpulan statement atau perintah untuk melakukan tugas tersebut.
4. Mengembalikan sebuah nilai kepada fungsi lain yang memanggil atau menggunakannya (jika perlu).

2. Fungsi di Dalam Bahasa Java

Dalam bahasa java terdapat 2 macam fungsi yaitu :

1. Fungsi yang mengembalikan/menghasilkan nilai (non void function)
2. Fungsi tidak mengembalikan/menghasilkan nilai (void function)

3. Deklarasi Function

Pada dasarnya, cara mendeklarasikan fungsi di dalam bahasa Java serupa dengan bahasa C. Berikut ini adalah cara mendeklarasikan fungsi di dalam bahasa Java:

<Modifier><Return type><function name><parameter list>

Dimana :

Modifier : sebuah *access modifier* (*public, private, protected*), yang dapat dikombinasikan dengan tipe *modifier* lain (*static, final, abstract*).

modifier static digunakan untuk mendefinisikan *member class*, sehingga metode/fungsi dapat diakses langsung dengan nama kelas (Untuk member/fungsi non static maka dipanggil melalui instant atau dengan membuat objek terlebih dahulu)

Return Type : type nilai pengembalian dari fungsi (*int, String, boolean* dll)

Function name : nama fungsi yang nantinya akan di panggil di dalam program

Parameter list : adalah **nama dan tipe variable** yang akan digunakan untuk menyimpan nilai yang dibutuhkan oleh fungsi tersebut. Jika terdapat lebih dari 1 parameter, maka parameter ditulis dipisahkan dengan koma.

Method tanpa parameter

Method tanpa parameter adalah method yang tidak memiliki parameter sama sekali.

Membuat dan memanggil fungsi

Fungsi didalam java dapat diletakkan disembarang tempat selama masih ada didalam class. Bisa sebelum program utama (public static void main) atau sesudahnya, Fungsi yang sudah dibuat dapat di panggil oleh fungsi yang lain atau dari dalam program utama.

Pemanggilan fungsi dilakukan dengan memanggil **nama fungsi** diikuti **parameternya (lihat contoh pada program)**.

a. Fungsi yang menghasilkan nilai (*Non Void Function*)

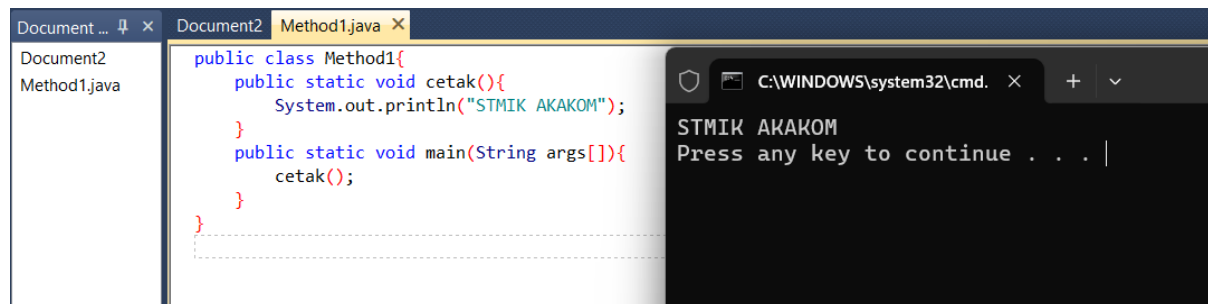
Adalah fungsi yang ketika kembali ke program utamanya disertai dengan membawa suatu nilai. Untuk mengembalikan nilai kedalam nama fungsi menggunakan perintah **return**.

b. Fungsi yang tidak menghasilkan nilai (*void Function*)

Sebuah fungsi tidak harus selalu mengembalikan nilai. Tipe dari fungsi yang tidak dapat mengembalikan nilai adalah **void**. Berikut adalah contoh fungsi yang tidak mengembalikan nilai, yaitu untuk menampilkan kata "Hello" ke layar sebanyak **n** kali.

C. PEMBAHASAN LISTING PRAKTIK

Praktik 1



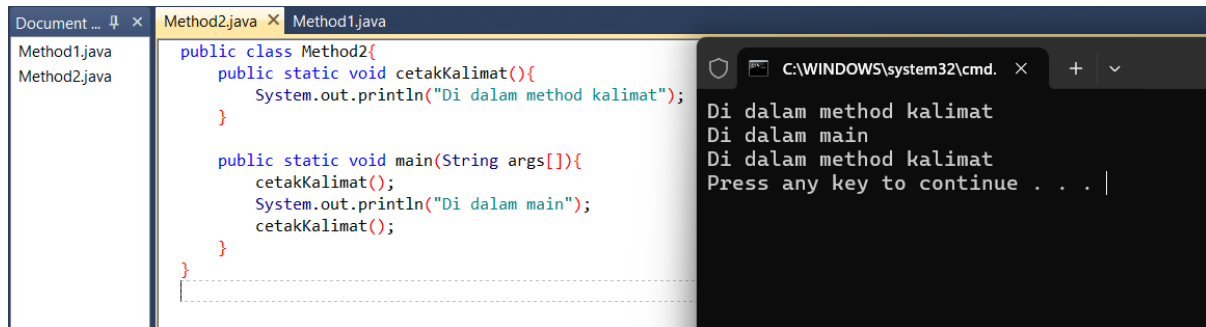
The screenshot shows an IDE with a file named 'Method1.java'. The code defines a class 'Method1' with two static methods: 'cetak()' which prints 'STMIK AKAKOM' to the console, and 'main(String args[])' which calls the 'cetak()' method. To the right, a terminal window shows the output of the program: 'STMIK AKAKOM' followed by a prompt 'Press any key to continue . . . |'.

```
public class Method1{
    public static void cetak(){
        System.out.println("STMIK AKAKOM");
    }
    public static void main(String args[]){
        cetak();
    }
}
```

```
STMIK AKAKOM
Press any key to continue . . . |
```

Pembahasan : Pada praktik pertama, program menampilkan cara pembuatan method sederhana tanpa parameter dan tanpa nilai balik. Di sini, dilakukan pemanggilan fungsi dengan memanggil nama fungsi yg sudah dibuat sebelumnya yaitu cetak pada public static void main(String args[]) maka akan mencetak yang ada di fungsi cetak yaitu STMIK AKAKOM ke layar.

Praktik 2



```
Document ... x Method2.java x Method1.java
Method1.java
Method2.java

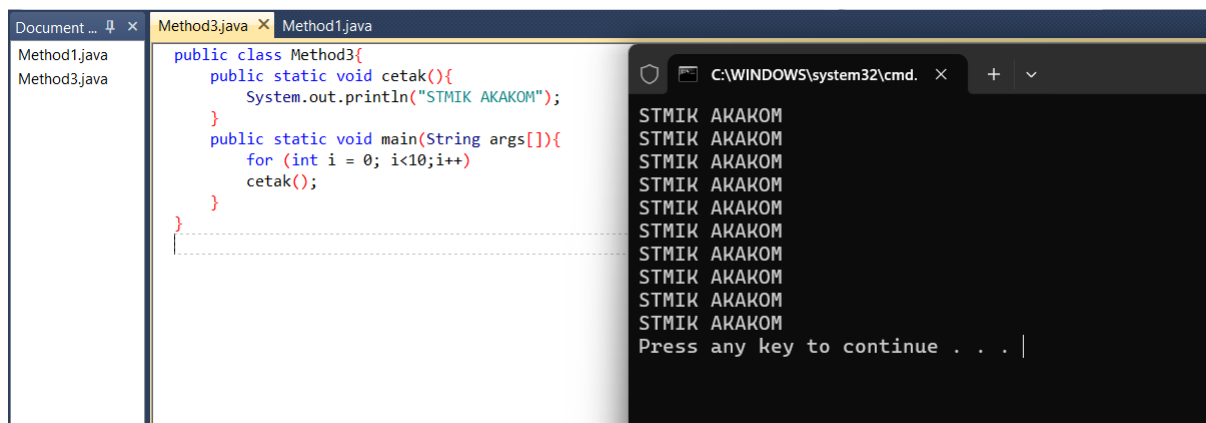
public class Method2{
    public static void cetakKalimat(){
        System.out.println("Di dalam method kalimat");
    }

    public static void main(String args[]){
        cetakKalimat();
        System.out.println("Di dalam main");
        cetakKalimat();
    }
}

C:\WINDOWS\system32\cmd. x + v
Di dalam method kalimat
Di dalam main
Di dalam method kalimat
Press any key to continue . . . |
```

Pembahasan : Praktik kedua mulai memperlihatkan bahwa method bisa dipanggil berkali-kali di dalam program. Method `cetakKalimat()` dibuat seperti praktik pertama, tetapi kali ini dipanggil dua kali: sebelum dan sesudah perintah `System.out.println("Di dalam main")`. Hasilnya, program akan mencetak isi method dua kali, dan ini membuktikan bahwa satu method bisa digunakan berulang selama masih berada dalam satu kelas. Ini juga mendukung konsep penghematan kode karena kita tidak perlu menulis ulang baris yang sama untuk setiap proses yang sama.

Praktik 3



```
Document ... x Method3.java x Method1.java
Method1.java
Method3.java

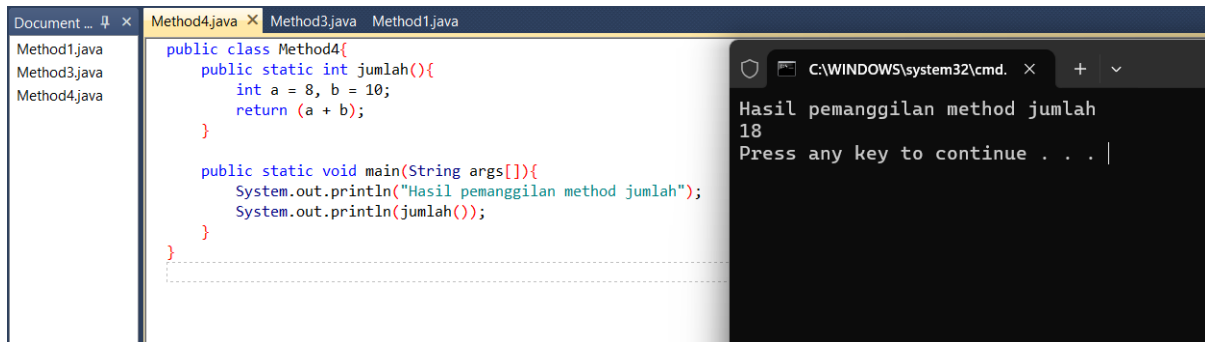
public class Method3{
    public static void cetak(){
        System.out.println("STMIK AKAKOM");
    }

    public static void main(String args[]){
        for (int i = 0; i<10;i++){
            cetak();
        }
    }
}

C:\WINDOWS\system32\cmd. x + v
STMIK AKAKOM
STMIK AKAKOM
STMIK AKAKOM
STMIK AKAKOM
STMIK AKAKOM
STMIK AKAKOM
STMIK AKAKOM
STMIK AKAKOM
STMIK AKAKOM
STMIK AKAKOM
Press any key to continue . . . |
```

Pembahasan : Praktik ketiga mengembangkan lagi praktik sebelumnya dengan memasukkan pemanggilan method ke dalam perulangan. Dalam program ini, method `cetak()` dipanggil sebanyak 10 kali di dalam sebuah `for`. Jadi output "STMIK AKAKOM" akan muncul 10 baris. Ini contoh yang sangat umum digunakan saat kita ingin menampilkan hasil yang sama berulang kali, dan menunjukkan bahwa method bisa diletakkan di mana saja selama berada dalam struktur yang benar. Perulangan juga membuktikan bahwa method tetap konsisten menghasilkan output yang sama tiap kali dipanggil.

Praktik 4



```
Document ... x Method4.java x Method3.java Method1.java
Method1.java
Method3.java
Method4.java

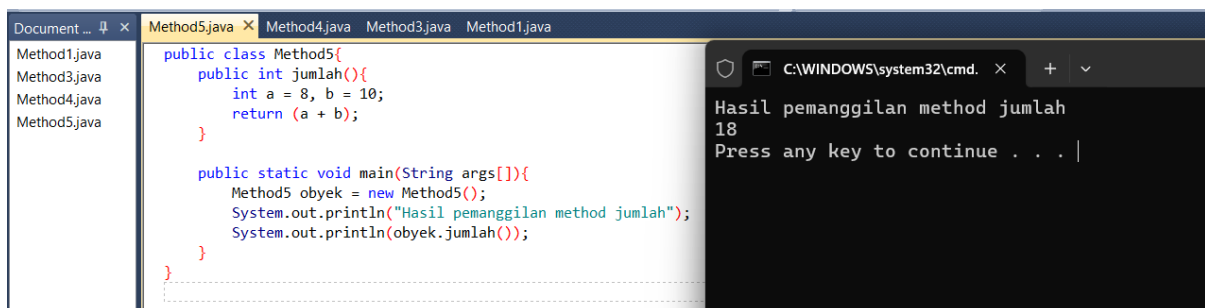
public class Method4{
    public static int jumlah(){
        int a = 8, b = 10;
        return (a + b);
    }

    public static void main(String args[]){
        System.out.println("Hasil pemanggilan method jumlah");
        System.out.println(jumlah());
    }
}

C:\WINDOWS\system32\cmd. x + v
Hasil pemanggilan method jumlah
18
Press any key to continue . . . |
```

Pembahasan : Pada praktik keempat, diperkenalkan method yang tidak hanya menjalankan perintah, tapi juga mengembalikan hasil ke pemanggilnya. Method jumlah() tidak menggunakan parameter tetapi di dalamnya terdapat proses penjumlahan dua bilangan, dan hasilnya dikembalikan ke main() menggunakan keyword return. Karena method bertipe int, maka nilai hasil penjumlahan bisa langsung dicetak di dalam System.out.println(). Ini penting karena dalam banyak kasus, program tidak hanya menampilkan tulisan, tapi juga perlu melakukan operasi dan mengembalikan hasilnya ke bagian utama program.

Praktik 5



```
Document ... x Method5.java x Method4.java Method3.java Method1.java
Method1.java
Method3.java
Method4.java
Method5.java

public class Method5{
    public int jumlah(){
        int a = 8, b = 10;
        return (a + b);
    }

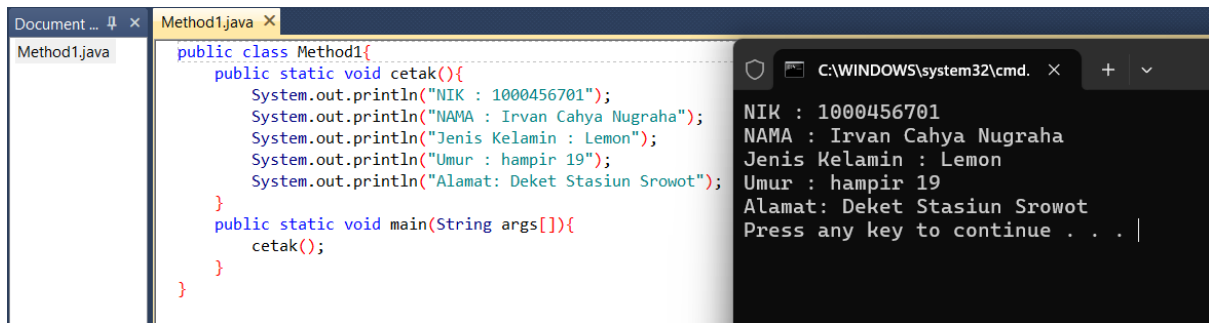
    public static void main(String args[]){
        Method5 obyek = new Method5();
        System.out.println("Hasil pemanggilan method jumlah");
        System.out.println(obyek.jumlah());
    }
}

C:\WINDOWS\system32\cmd. x + v
Hasil pemanggilan method jumlah
18
Press any key to continue . . . |
```

Pembahasan : Praktik kelima memperkenalkan konsep objek dengan method yang tidak menggunakan kata static. Karena method jumlah() di sini tidak static, maka pemanggilannya tidak bisa dilakukan langsung. Program harus terlebih dahulu membuat objek dari kelas Method5, lalu melalui objek tersebut memanggil method-nya. Ini memperkenalkan konsep dasar pemrograman berorientasi objek, di mana method-method yang bukan static hanya bisa diakses lewat objek. Struktur kode juga tetap sama dengan praktik keempat, hanya perbedaannya ada pada cara pemanggilan dan penggunaan objek sebagai penghubung.

LATIHAN

Latihan 1



```
public class Method1{
    public static void cetak(){
        System.out.println("NIK : 1000456701");
        System.out.println("NAMA : Irvan Cahya Nugraha");
        System.out.println("Jenis Kelamin : Lemon");
        System.out.println("Umur : hampir 19");
        System.out.println("Alamat: Deket Stasiun Srowot");
    }
    public static void main(String args[]){
        cetak();
    }
}
```

NIK : 1000456701
NAMA : Irvan Cahya Nugraha
Jenis Kelamin : Lemon
Umur : hampir 19
Alamat: Deket Stasiun Srowot
Press any key to continue . . . |

Pembahasan : Latihan pada modul ini meminta untuk membuat method yang menerima dua bilangan sebagai parameter, lalu mengembalikan hasil operasinya ke method main. Dalam program yang dibuat, method diberi nama penjumlahan, yang memiliki dua parameter bertipe int. Method ini kemudian mengembalikan hasil dari penjumlahan kedua bilangan tersebut menggunakan keyword return.

Di dalam method main, user diminta untuk memasukkan dua angka melalui Scanner, lalu kedua angka itu dikirim ke method penjumlahan. Hasil yang dikembalikan oleh method tersebut disimpan ke dalam variabel dan langsung ditampilkan ke layar. Ini sekaligus menunjukkan alur data: input dari user, diproses di method, lalu dikembalikan lagi ke program utama.

TUGAS

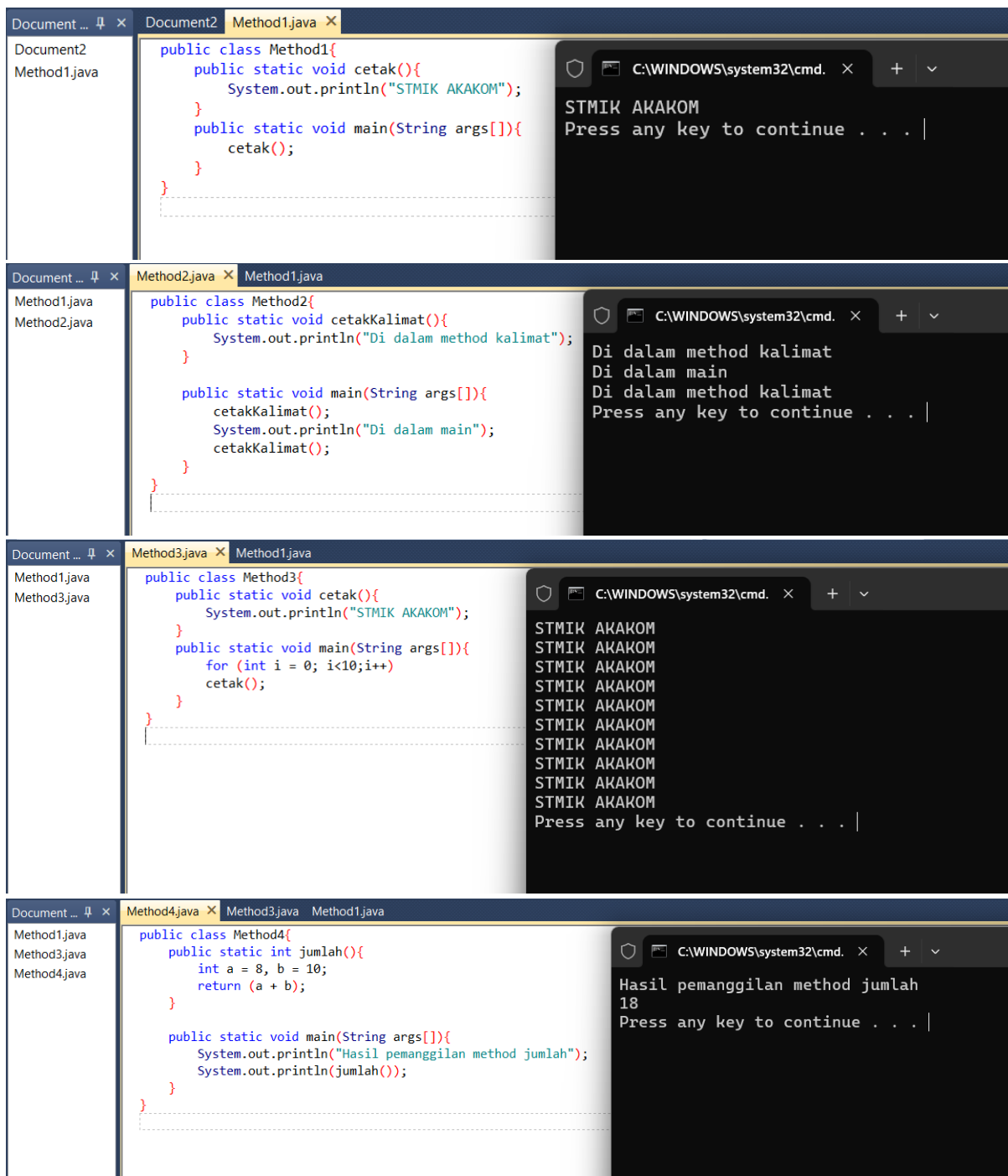
Tugas 1

D. PEMBAHASAN TUGAS

E. KESIMPULAN

Dari modul ini saya belajar tentang cara membuat dan menggunakan method di dalam program Java. Dengan method, program jadi lebih ringkas, terstruktur, dan mudah dibaca. Saya juga paham bahwa method bisa dibuat dengan atau tanpa parameter, serta bisa punya nilai kembalian ataupun tidak. Konsep seperti parameter, return value, dan overloading

F. LAMPIRAN



```
Document ... x Method5.java x Method4.java Method3.java Method1.java
Method1.java
Method3.java
Method4.java
Method5.java

public class Method5{
    public int jumlah(){
        int a = 8, b = 10;
        return (a + b);
    }

    public static void main(String args[]){
        Method5 obyek = new Method5();
        System.out.println("Hasil pemanggilan method jumlah");
        System.out.println(obyek.jumlah());
    }
}
```

```
C:\WINDOWS\system32\cmd. x + v
Hasil pemanggilan method jumlah
18
Press any key to continue . . . |
```

```
Document ... x Method1.java x
Method1.java

public class Method1{
    public static void cetak(){
        System.out.println("NIK : 1000456701");
        System.out.println("NAMA : Irvan Cahya Nugraha");
        System.out.println("Jenis Kelamin : Lemon");
        System.out.println("Umur : hampir 19");
        System.out.println("Alamat: Deket Stasiun Srowot");
    }

    public static void main(String args[]){
        cetak();
    }
}
```

```
C:\WINDOWS\system32\cmd. x + v
NIK : 1000456701
NAMA : Irvan Cahya Nugraha
Jenis Kelamin : Lemon
Umur : hampir 19
Alamat: Deket Stasiun Srowot
Press any key to continue . . . |
```